資料結構
Data Structure

Lab 04

姓名 : Name (盧敬恩)
學號 : Student ID (114310123)

Lab04-Q1. (完成Q1中的Call by value與Call by address的轉換)

更改Q1程式更改成call by address

Code

```
#include <iostream>
#include <string>
using namespace std;
struct Student {//宣告結構student 有id/名稱/國英數社自成績
    int id;
    string name;
    int chinese;
    int english;
    int math;
    int social;
    int science;
};
void updateMath(Student *s) {//傳址呼叫 把數學成績設定為100分
    s->math = 100;
}
void printStudent(Student s) { //該函式輸出學生的id 姓名 數學成績
    cout << "學生編號: " << s.id << endl;
    cout << "姓名: " << s.name << endl;
    cout << "數學成績: " << s.math << endl;
}
int main() {
    Student one; //宣告結構 student one 並賦值成績/姓名
    one.id = 1;
    one.name = "張小美";
    one.chinese = 77;
    one.english = 83;
    one.math = 86;
    one.social = 68;
    one.science = 91;

    updateMath(&one); //用函式將結構one的數學成績設為100
    printStudent(one); //輸出學生的id 姓名 數學成績

    return 0;
}
```

Result

Discussion

(.):傳值呼叫，當進行如`int a = one.math=100;`時
會先在記憶體內開一次one.math的副本,這一動作就是傳值呼叫，更改的僅是副本
(->) :傳址呼叫，這裡更改的是傳入的"地址"的內容，不會建立副本，而是更改該位址的數值。以下一範例可以說明此現象

ConsoleApplication1

```
#include <iostream>
#include <string>
using namespace std;
struct Student {//宣告結構student 有id ， 名稱，國英數社自成績
    int id;
    string name;
    int chinese;
    int english;
    int math;
    int social;
    int science;
};
void updateMath(Student* s) {//傳址呼叫 把數學成績設定為100分
    s->math = 100;
}
void updateMath_1(Student s) {//傳址呼叫 把數學成績設定為100分
    int a = s.math = 99;
    printf_s("a = %d",a);
}


void printStudent(Student s) { //該函式輸出學生的id 姓名 數學成績
    cout << "學生編號: " << s.id << endl;
    cout << "姓名: " << s.name << endl;
    cout << "數學成績: " << s.math << endl;
}
int main() {
    Student one; //宣告結構 student one 並賦值成績/姓名
    one.id = 1;
    one.name = "張小美";
    one.chinese = 77;
    one.english = 83;
    one.math = 86;
    one.social = 68;
    one.science = 91;

    updateMath(&one); //用函式將結構one的數學成績設為100
    printStudent(one);//先輸出學生的id 姓名 數學成績
    updateMath_1(one);//用傳值呼叫
    printStudent(one); //輸出學生的id 姓名 數學成績

    return 0;
}
```

這裡有兩個用於更改的副程式,

```
void updateMath(Student* s)  //傳址呼叫
void updateMath_1(Student s)//傳值呼叫
```

該程式會先進行一次傳址呼叫, 並輸出
在進行一次傳值呼叫, 並輸出
updateMath_1 裡面的 int a用以證明更改副本的推論

從運行結果中可以看到一開始傳址呼叫成功把數學成績改成100了
接著, ***void updateMath_1***(Student s)使用傳值呼叫成功更改了"副本"
但是最後輸出時然沒有把成績設為99。
說明該位址的值還是一開始**void *updateMath***更改的100